

重大 AI 更新提醒

10-04 | llama.cpp 修复 CUDA MMQ 在专家数远超微批时的内存故障

本期导读

本期聚焦 llama.cpp 的最新官方 release b11390：该版本修复了 CUDA 后端在 MoE 模型推理中，当专家数量（`n_expert`）远大于微批大小（`n_ubatch`）时 MMQ 内核可能触发的内存访问故障。对于在本地或服务器上以 CUDA 运行 MoE 大模型的开发者而言，这是一次值得尽快跟进的稳定性修复。

已检查来源

GitHub Release

1. llama.cpp b11390 发布：修复 CUDA MMQ 在 `n_expert` 远大于 `n_ubatch` 时的内存故障

来源 GitHub Release | 类型 GitHub Release | 主题 检索增强与向量模型 时间 10-04 07:52 | 分数 7

是什么 这是 ggml-org/llama.cpp 的官方 GitHub Release b11390，核心变更是对 CUDA 后端 MMQ（Matrix Multiplication Quantized，量化矩阵乘法）内核的修复：当模型的专家数量 `n_expert` 远大于推理微批大小 `n_ubatch` 时，MMQ 可能发生内存访问故障。该 release 同时提供 macOS Apple Silicon（arm64）、macOS Intel（x64）、iOS XCFramework、Ubuntu x64/arm64/s390x（CPU）等预编译产物下载。

通俗解释 可以把 MoE（混合专家）模型想象成一家拥有很多专科医生的大医院：每次来一小批病人（`n_ubatch` 很小），但医院里专科医生数量极多（`n_expert` 很大）。llama.cpp 的 CUDA 加速模块在安排 哪位医生看哪位病人 时，原本的排班表在小批量、多专家的情况下会算错地址，导致程序去读不该读的内存区域，轻则报错崩溃，重则结果异常。b11390 修正了这段排班逻辑，让它在专家远多于当前批次时也能安全地完成量化矩阵乘法计算。

主要作用 解决在 CUDA 上运行 MoE 类大模型时，因专家数量与微批大小比例悬殊而触发的 MMQ 内存故障，提升本地推理与批量推理的稳定性，避免崩溃或错误输出。对于依赖 llama.cpp 做量化模型部署的团队，这是一个应当尽快升级的修复版本。

核心功能 1) 修复 CUDA MMQ 内核在 `n_expert` 远大于 `n_ubatch` 场景下的内存访问故障；2) 覆盖 MoE 模型在低微批、高专家数配置下的推理路径；3) 随 release 提供 macOS（Apple Silicon 与 Intel）、iOS XCFramework、Ubuntu x64/arm64/s390x 等多平台预编译包；4) 属于 llama.cpp 持续迭代的官方构建，便于直接替换升级。

对比判断 候选摘要未提供修复前后的 benchmark、吞吐或延迟对比数据，暂无可信公开对比数据。

适合谁 适合在本地或服务器上使用 CUDA 运行 llama.cpp 的开发者、负责 MoE 大模型量化部署的工程团队，以及关注 ggml/llama.cpp 生态、需要跟进稳定性修复的研究者与运维人员。

直达链接 <https://github.com/ggml-org/llama.cpp/releases/tag/b11390>

本期仅收录 1 条候选，内容基于 GitHub Release 官方说明；未提供的性能数据不作推测。